PROJE RAPORU

TURUNCU PROJE

TAMAMLANDI

Malzahar

Yazar(lar):

Famura

İçerikler

1. Giriş 3
2. Proje Tanımı 3
3. Proje Teknolojileri 3
   3.1. Modüller 3
   3.2. Programlama Dilleri 4
   3.3. Kütüphaneler ve Harici Bileşenler 4
4. Proje Mimarisi 4
   4.1. Genel Mimari 4
   4.2. Güvenlik Modeli 4
      4.2.1. İstek Gizliliği (Client → Server control payload) 5
      4.2.2. İstek Bütünlüğü (Client → Server) 5
      4.2.3. Yanıt Doğrulama (Server → Client) 6
      4.2.4. Büyük Veri Şifreleme (ZIP chunks) 6
   4.3. İstek / Yanıt Akışı 7
      4.3.1. Genel İşlem Sırası: 7
      4.3.2. Yanıt Tarafında: 7
   4.4. Komut Satırı Arayüzü (CLI) 7
   4.5. Klasör Eşleme 8
   4.6. ZIP İşleme 8
   4.7. Parçalı Transfer (Push / Pull) 9
      4.7.1. Push: 9
      4.7.2. Pull: 9
5. Proje Analizi 10
   5.1. Test 10
   5.2. Geliştirilebilir Alanlar 10
6. Proje Yorumları 11
   6.1. Olası Kullanım Alanları 11
   6.2. Kullanım Notları 11
   6.3. Geliştirici YorumuCreator Comments 11
7. Credits 12
8. Kaynaklar 12
9. Disk İçeriği 12

1. Giriş

Malzahar projesi, günlük kullanım için basit dosyalar üzerinde **git benzeri bir altyapıyı anlamak ve geliştirmek amacıyla** oluşturulmuş özel bir projedir. Raporun geri kalan kısmı projenin teknik yönlerine odaklanırken, bu giriş bölümü projenin ortaya çıkış amacını ve günlük kullanım senaryosunu açıklamaktadır.

Proje, kişisel bir ihtiyaçtan doğdu aslında. LA Noire oyununu iki farklı bilgisayarda oynarken kayıt dosyalarını senkronize etmek için bir **push / pull mekanizmasına** ihtiyaç duydum. Steam ve Rockstar Launcher bu kayıt dosyalarını otomatik olarak paylaşmıyordu. Steam üzerinden bazı çözümler mevcut olsa da, kendi web sitem (Nexus) üzerinde çalışan bir sistem kurmanın iyi bir proje olacağını düşündüm. Bu nedenle sistemi geliştirdim ve şu anda sorunsuz şekilde çalışıyor.

Not: Bu rapor yalnızca **Malzahar'ın istemci tarafını** kapsıyor. Projenin tamamının detaylı yapısını anlamak için **Nexus raporuna** da bakılmasını öneririm.

2. Proje Tanımı

Malzahar, Windows üzerinde çalışan güvenli bir dosya ve klasör senkronizasyon istemcisi olarak tasarlandı. İstemci, HTTPS üzerinden uzak bir sunucu ile iletişim kuruyor. Program bir komut satırı aracı olarak çalışır ve şu işlemleri gerçekleştirir:

- **K**ullanıcı Doğrulama (login)
- Yerel Klasörü Sunucuya Bağlama (connect)
- Sunucudaki Verileri Kontrol Etme (check)
- Sunucudan Klasör İndirme (pull)
- Yerel Klasörü Sunucuya Yükleme (push)
- Sunucu Üzerindeki Versiyonları Yönetme (manage)

Sistemi tasarlarken üç temel hedef belirledim:

- **Gizlilik:** İstemci ve sunucu arasındaki veriyi RSA ve AES-GCM ile şifreleniyor.
- **Bütünlük:** Her isteğe HMAC-SHA256 imzası ekleniyor.
- **Büyük Veri Transferi:** ZIP arşivleme ve parçalı veri transferi kullanılıyor.

Malzahar, kişisel kullanım için geliştirdiğim bir altyapı aracıdır. Aynı zamanda portfolyomdaki teknik projelerden biridir.

3. Proje Teknolojileri

3.1. Modüller

- **Windows (Windows 11):**
  Projeyi Windows üzerinde çalışacak şekilde tasarladım. Ağ iletişimi, kriptografi, dosya sistemi işlemleri ve süreç yönetimi için Windows'un yerel API'lerini kullandım. Projeyi Windows 7 veya Windows 10 üzerinde test etmedim.

3.2. Programlama Dilleri

- **C++ (C++14 Standard) – Visual Studio 2022:**
  Projenin tamamını C++ ile yazdım. Derleme sürecini CMake ile yönetiyorum ve C++14 standardını kullanıyorum.

3.3. Kütüphaneler ve Harici Bileşenler

- **Windows / System:**
  - **WinHTTP (winhttp.h):** Sunucu ile HTTPS iletişimini WinHTTP ile kuruyorum.
  - **bcrypt (bcrypt.h):** AES-GCM şifreleme, rastgele sayı üretimi ve HMAC işlemlerini bcrypt API ile gerçekleştiriyorum.
  - **Crypt32 (wincrypt.h / crypt32.lib):** Base64 kodlama/çözme işlemleri ve RSA public key import işlemleri için Crypt32 kullanıyorum.
  - **std::filesystem:** Klasör eşleme, dizin temizleme ve geçici dosya işlemlerini filesystem ile yönetiyorum.

- **OpenSSL:**
  - **openssl/evp.h, openssl/pem.h, openssl/rsa.h:** RSA OAEP ile sunucu yanıtlarını çözmek için OpenSSL kütüphanesini kullanıyorum.

- **Windows / Powershell:**
  - Klasörden ZIP oluşturmak için **Compress-Archive**.
  - ZIP'leri açıp hedef klasöre atmak için **Expand-Archive**.

4. Proje Mimarisi

4.1. Genel Mimari

Malzahar bir CLI istemcisi olarak çalışır ve HTTPS endpointleri üzerinden sunucu ile iletişim kuruyor.

Kullandığım temel endpointler:

- /api/private/transfer (control channel: auth + commands)
- /api/private/transfer/pushzip (chunk upload endpoint)
- /api/private/transfer/zipchunk (chunk download endpoint)

İstemci iki farklı veri transfer modeli kullanıyor:

- **Push:** local klasör→ ZIP → AES-GCM chunk şifreleme→ chunkları yükle
- **Pull:** AES anahtarını ve chunk planı iste → şifrelenmiş chunkları indir→ AES-GCM şifre çözmesi→ ZIP'leri açıp klasöre yükle

4.2. Güvenlik Modeli

Malzahar çok katmanlı bir güvenlik mimarisi kullanıyor:

4.2.1. İstek Gizliliği (Client → Server control payload)

- İstemci tarafında bir JSON veri paketi oluşturuyor. Bu paket şu alanları barındıyor:
  - request (login/connect/check/pull/push/manage)
  - name, password
  - data (scope/folder/version/key etc.)

```
std::string build_signed_request_payload(const std::string& request, const std::string& name,
    const std::string& password, const std::string& data) {
    std::string json = "{\"request\":\"" + request + "\",\"name\":\"" + name +
        "\",\"password\":\"" + password + "\",\"data\":\"" + data + "\"}";

    std::vector<unsigned char> spki = base64_decode(RSA_PUBLIC_KEY_B64);
    std::vector<unsigned char> encrypted = rsa_encrypt_oaep_spki(spki,
        std::vector<unsigned char>(json.begin(), json.end()));
    std::string encryptedB64 = base64_encode(encrypted);

    return build_signed_payload(encryptedB64);
}
```

- JSON verisini sunucunun public key'i ile **RSA OAEP kullanarak şifreliyor.**

```
    BCRYPT_OAEP_PADDING_INFO paddingInfo;
    paddingInfo.pszAlgId = BCRYPT_SHA256_ALGORITHM;
    paddingInfo.pbLabel = nullptr;
    paddingInfo.cbLabel = 0;

    DWORD outLen = 0;
    NTSTATUS status = BCryptEncrypt(hKey, (PUCHAR)data.data(), (ULONG)data.size(),
        &paddingInfo, nullptr, 0, nullptr, 0, &outLen, BCRYPT_PAD_OAEP);
    std::vector<unsigned char> out(outLen);
    status = BCryptEncrypt(hKey, (PUCHAR)data.data(), (ULONG)data.size(),
        &paddingInfo, nullptr, 0, out.data(), (ULONG)out.size(), &outLen, BCRYPT_PAD_OAEP);

    BCryptDestroyKey(hKey);
    LocalFree(pInfo);

    out.resize(outLen);
    return out;
}
```

- Ardından Base64 formatına çeviriyor.

```
std::string base64_encode(const std::vector<unsigned char>& data) {
    DWORD outLen = 0;
    CryptBinaryToStringA(data.data(), (DWORD)data.size(),
        CRYPT_STRING_BASE64 | CRYPT_STRING_NOCRLF, nullptr, &outLen);
    std::string out(outLen, '\0');
    CryptBinaryToStringA(data.data(), (DWORD)data.size(),
        CRYPT_STRING_BASE64 | CRYPT_STRING_NOCRLF, &out[0], &outLen);
    if (!out.empty() && out.back() == '\0') out.pop_back();
    return out;
}
```

4.2.2. İstek Bütünlüğü (Client → Server)

- Şifrelenmiş verinin üzerine **HMAC-SHA256 imzası ekliyor**.

```
std::string build_signed_payload(const std::string& encryptedBase64) {
    auto sig = hmac_sha256(g_hmac_key, encryptedBase64);
    std::string sigB64 = base64_encode(sig);
    return encryptedBase64 + "|" + sigB64;
}
```

- Gönderdiğim veri şu formatta: encryptedBase64 | signatureBase64

4.2.3. Yanıt Doğrulama (Server → Client)

- HMAC imzası doğrulanır.

```
std::string encryptedB64 = signedPayload.substr(0, sep);
std::string signatureB64 = signedPayload.substr(sep + 1);
std::string expectedSignatureB64 = base64_encode(hmac_sha256(g_hmac_key, encryptedB64));

if (!constant_time_equal(signatureB64, expectedSignatureB64)) {
    throw std::runtime_error("Response signature verification failed");
}
```

- **RSA OAEP** ile veri çözülür. Son olarak parse edilir.

```
std::vector<unsigned char> hashObject(hashObjectSize);
std::vector<unsigned char> hash(hashLen);

if (BCryptCreateHash(hAlg, &hHash, hashObject.data(), hashObjectSize,
    (PUCHAR)key.data(), (ULONG)key.size(), 0) != 0)
    throw std::runtime_error("BCryptCreateHash failed");

BCryptHashData(hHash, (PUCHAR)data.data(), (ULONG)data.size(), 0);
BCryptFinishHash(hHash, hash.data(), (ULONG)hash.size(), 0);

BCryptDestroyHash(hHash);
BCryptCloseAlgorithmProvider(hAlg, 0);

return hash;
```

4.2.4. Büyük Veri Şifreleme (ZIP chunks)

- Büyük veri transferleri için **AES-256-GCM** kullanılıyor.
- Veri yaklaşık **80 MB'lık parçalara** bölünür (Nexus'u yayınlamak için kullanılan cloudflare 100 mb'ye kadar izin veriyor. Güvenli olsun diye 80 seçtim ben yine de).
- Her parça:
  - random 12-byte IV
  - 16-byte GCM tag

  içerir.

```
status = BCryptEncrypt(hKey,
    (PUCHAR)plain.data(), (ULONG)plain.size(),
    &authInfo,
    nullptr, 0,
    cipher.data(), (ULONG)cipher.size(), &cbCipher, 0);

BCryptDestroyKey(hKey);
BCryptCloseAlgorithmProvider(hAlg, 0);

if (status != 0) {
    throw std::runtime_error("BCryptEncrypt(GCM) failed");
}
cipher.resize(cbCipher);
```

```
cipher.resize(cbCipher);

return base64_encode(iv) + "|" + base64_encode(cipher)
    + "|" + base64_encode(tag);
```

Şifrelemeden sonuç şu şekilde çıkar:

- IV + Ciphertext + Tag

Bu modelde, Kontrol düzlemi için RSA + HMAC ve Veri düzlemi için AES-GCM kullanılır.

4.3. İstek / Yanıt Akışı

Komutlar gönderim yapmak için ortak bir gönderme fonksiyonu kullanıyorlar. Buraya verilerini iletiyorlar, bu fonksiyon da tek şifreli bir payload haline getirip HTTPS POST ile gönderiyor.

4.3.1. Genel İşlem Sırası:

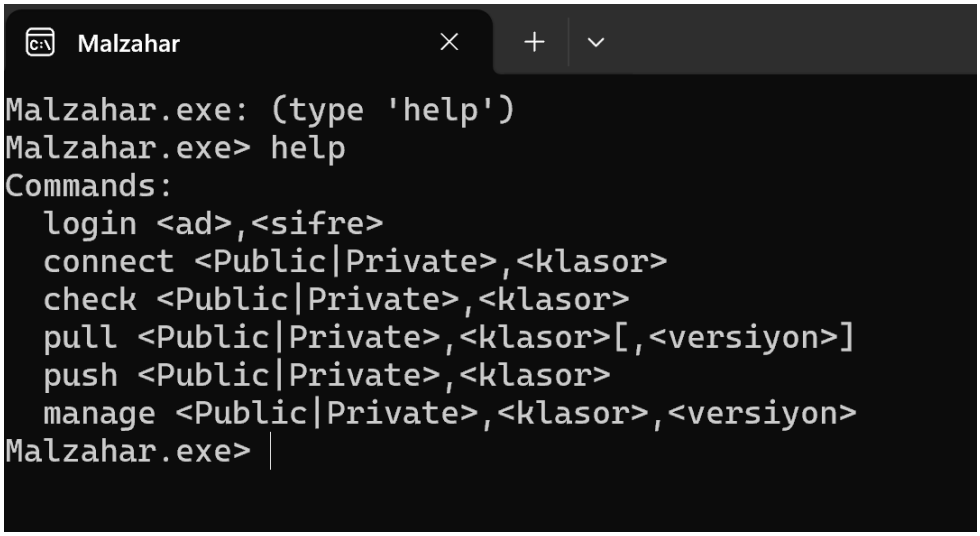
1) JSON veri oluşturulur
2) RSA ile şifrelenir
3) Base64 kodlanır
4) HMAC imzası oluşturulur
5) HTTPS POST isteği gönderilir

4.3.2. Yanıt Tarafında:

1) Veri ve imza ayrılır
2) HMAC doğrulanır
3) RSA ile çözülür
4) JSON parse edilir

4.4. Komut Satırı Arayüzü (CLI)

Malzahar aslında interaktif bir komut satırı program olarak çalışır:



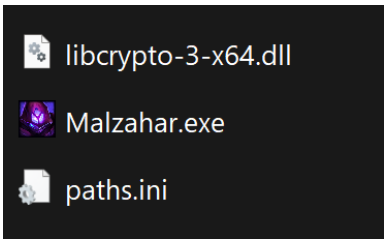
Desteklenen Komutlar:

- login <name>,<password> / Data'yı servera gönderir ve kabul etmesi durumunda local olarak RAM'de tutar.
- connect <Public|Private>,<folder>,<path> / Serverdaki klasörlerle local klasörleri eşleştirir.
- check <Public|Private>,<folder> / Bir entry için var olan tüm versionları gösterir.
- pull <Public|Private>,<folder>[,<version>] / Server'daki versyonları çekmemizi sağlar
- push <Public|Private>,<folder> / Sizdeki versiyonu server'a yükler
- manage <Public|Private>,<folder>,<version> / Herhangi bir versiyonu silmenizi sağlar

Not:

- Çoğu işlem için kullanıcı girişinin yapılmış olması gerekir.
- Klasör işlemleri daha önceden paths.ini'ye klasör eşleme yapılmış olmasını gerektiriyor. (connect ile yapılabilir).

4.5. Klasör Eşleme



Malzahar **paths.ini** adlı bir konfigürasyon dosyası kullanır. Her girişin formatı Scope,Folder-Path şeklindedir. Örnek olarak; Private,Documents-C:\Users\Me\Documents\ProjectX.
Kullanıcı connect komutunu çalıştırdığında Malzahar:

- scope/folder bilgisini sunucuya gönderir
- işlem başarılı olursa yerel klasör eşlemesini paths.ini dosyasına kaydeder
- daha sonra gelen komutlar (check, pull, push, manage) klasör yolunu bu dosyadan bulur (büyük/küçük harf duyarsız eşleşme desteklenir)

Böylece, CLI kullanımını hızlı ve kullanıcıyı yerel klasör yolunu sürekli tekrar yazmaktan kurtarıyor.

4.6. ZIP İşleme

Malzahar klasörleri ZIP arşivine çevirerek transfer eder..

Push Sırasında:

- Sistemin geçici dizininde geçici bir ZIP arşivi oluşturur
- ZIP oluşturmak için PowerShell Compress-Archive komutunu kullanır
- Klasör boşsa ZIP oluşturma işlemini hata olarak değerlendirir

```
if (!create_zip_with_powershell(sourcePath, tempZip)) {
    std::cout << "push failed: zip create failed (folder may be empty)\n";
    continue;
}

std::cout << "push: preparing aes key ...\n";
std::vector<unsigned char> aesKey = generate_aes_key_256();
std::string aesKeyB64 = base64_encode(aesKey);

// server'a canonical (ini'deki) scope/klasor gönder
std::string pushData = entry.scope + "/" + entry.klasor + "/" +
    aesKeyB64;
std::string pushControlPayload = build_signed_request_payload("push",
    logined_ad, logined_sifre, pushData);

std::cout << "push: sending control payload ...\n";
std::string controlResp = trim(http_post_raw(TRANSFER_PATH, pushControlPayload));
if (controlResp != "0") {
    std::cout << "push failed: control step rejected, "
        << format_server_response(controlResp)
        << "\n";
    std::error_code ec;
    std::filesystem::remove(tempZip, ec);
    continue;
}
                std::cout << "push: control ok, uploading zip in chunks ...\n";
const size_t CHUNK_SIZE = 80ull * 1024ull * 1024ull; // 80 MiB plaintext chunk
bool ok = push_zip_in_chunks(tempZip, aesKey, CHUNK_SIZE);
```

Pull Sırasında:

- ZIP dosyasını indiririm
- Yerel klasöre çıkarırım
- Eski klasör içeriğini temizlerim

```
bool ok = pull_zip_in_chunks(tempZip, aesKey, totalChunks);
if (!ok) {
    std::error_code ec;
    std::filesystem::remove(tempZip, ec);
    std::cout << "pull failed: chunk download aborted\n";
    continue;
}

bool extracted = extract_zip_with_powershell(tempZip, targetPath);
std::error_code ec;
std::filesystem::remove(tempZip, ec);
```

4.7. Parçalı Transfer (Push / Pull)

4.7.1. Push:

Push işlemi iki aşamadan oluşuyor:

Aşama A —Kontrol

- Yeni bir AES anahtarı üretilir ve sunucuya gönderilir.
- Sunucu başarı kodu (0) döndürürse transferi başlatılır.

```
std::filesystem::path sourcePath = entry.path; // canonical path
if (!std::filesystem::exists(sourcePath) || !std::filesystem::is_directory(sourcePath)) {
    std::cout << "push failed: source path is missing or not a directory\n";
    continue;
}

std::cout << "push: creating zip from " << sourcePath.string() << " ...\n";
auto now = std::chrono::steady_clock::now().time_since_epoch().count();
tempZip = std::filesystem::temp_directory_path() /
    ("malzahar_push_" + std::to_string(now) + ".zip");
```

Aşama B — Veri Yükleme

- ZIP dosyasını parçalara bölünür.
- Her parçayı AES-GCM ile şifrelerim ve sunucuya gönderilir.
- Her parça için en fazla 3 tekrar denemesi yapılır.

```
std::cout << "push: control ok, uploading zip in chunks ...\n";
const size_t CHUNK_SIZE = 80ull * 1024ull * 1024ull; // 80 MiB plaintext chunk
bool ok = push_zip_in_chunks(tempZip, aesKey, CHUNK_SIZE);

std::error_code ec;
std::filesystem::remove(tempZip, ec);

if (!ok) {
    std::cout << "push failed: chunk upload aborted\n";
    continue;
}

std::cout << "push ok: uploaded successfully\n";
```

4.7.2. Pull:

Pull işleminde sunucu şu bilgileri döndürür:

- response: "0"
- aes_key: "<base64>"
- total_chunks: <int>

İstemci ondan sonra şu adımları izler:

- Her chunk için index:
  - İstemci [idx][total] ‘u /zipchunk a POST’lar
  - Server [idx][total] + IV + ciphertext + TAG bilgisini döner
  - İstemci şifreyi çözer ve onları plaintext halinde temp’e zip olarak yazar
- Tüm chunklar bittikten sonra:
  - Tüm ZIP2leri ve temp dosyasını düzenler
- Her chunk için 3 deneme hakkı vardır. Üçüncü de başarısız olursa işlem durdurulur.

```
std::cout << "pull: downloading zip in chunks ... total=" << totalChunks << "\n";
bool ok = pull_zip_in_chunks(tempZip, aesKey, totalChunks);
if (!ok) {
    std::error_code ec;
    std::filesystem::remove(tempZip, ec);
    std::cout << "pull failed: chunk download aborted\n";
    continue;
}

bool extracted = extract_zip_with_powershell(tempZip, targetPath);
std::error_code ec;
std::filesystem::remove(tempZip, ec);

if (!extracted) {
    std::cout << "pull failed: zip extraction failed\n";
    continue;
}

std::string rootFolder = flatten_single_root_folder_if_needed(targetPath);
std::string versionInfo = version.empty() ? (rootFolder.empty() ? "latest" : rootFolder) : version;
```

5. Proje Analizi

5.1. Test

Proje Windows üzerinde çalışacak şekilde tasarlanmıştır ve **yerel Windows bileşenlerini** kullanır (WinHTTP + BCrypt + Crypt32). Transfer mekanizmasını tasarlarken ise şu özelliklere odaklandım:

- Uzun süreli yükleme/indirme işlemleri için geniş zaman aşımı süreleri (yaklaşık 10 dakikaya kadar)
- Her veri parçası için yeniden deneme mekanizması (3 tekrar)
- Veri gizliliği ve bütünlüğünü sağlamak için AES-GCM şifreleme

Yapılan Testler:

- Başarılı / başarısız giriş testleri (yanlış parola, kullanıcı bulunamadı)
- Büyük veri içeren klasör yükleme testi (1 GB üzeri) — parçalama ve yeniden deneme mekanizmasını doğrulamak için
- Büyük klasör indirme testi ve ZIP açma sonrası birebir veri eşleşmesi doğrulaması
- Veri aktarımı sırasında ağ kesintisi simülasyonu ve yeniden deneme davranışının doğrulanması

5.2. Geliştirilebilir Alanlar

Malzahar'ı daha ileri bir noktaya taşımak için aşağıdaki alanlarda geliştirme yapabilir:

- **Kaldığı Yerden Devam Eden Transfer Sistemi:**
    - Şu an bir parça üç kez üst üste başarısız olursa aktarımı tamamen durduyor.
    - Daha gelişmiş bir yapıda son başarıyla tamamlanan parçayı kaydedip transferi en baştan başlatmadan devam ettirilebilir.
- **Performans**
    - Şu an parçaları sırayla gönderiyor ve sırayla indiriyorum.
    - İleride birden fazla parçayı aynı anda işleyen paralel yapılı bir sistem kurulabilir.

- **Daha Sağlam bir JSON Parser Kullanımı:**
    - Şu an bazı yanıtları basit anahtar-değer yapıları için regex ile ayrıştırıyorum. Bu yöntem küçük ve kontrollü veri yapılarında yeterli olsa da daha karmaşık JSON yapılarında hata riskini artırır.
    - İleride özel bir JSON kütüphanesi kullanarak hem kodu daha temiz hale getirebilir hem de ayrıştırma güvenilirliğini artırılabilir.
- **Daha Ayrıntılı Hata Cevapları**
    - Şu an istemci bazı hata durumlarını yalnızca temel kodlarla yorumluyor.
    - Daha ayrıntılı sunucu yanıtları ekleyerek hatanın hangi aşamada oluştuğunu daha net görebilirim.
- **Platform Desteği**
    - Şu an sistem Windows'a bağımlı çalışıyor.
    - Bunun temel sebebi WinHTTP, BCrypt ve PowerShell tabanlı ZIP işlemleri kullanmam.
    - İleride ağ iletişimi, kriptografi ve arşivleme taraflarını platformdan bağımsız kütüphanelerle değiştirerek Linux ve macOS ve hatta belki Android desteği de ekleyebilirim.

6. Proje Yorumları

6.1. Olası Kullanım Alanları

- Bilgisayarlar arasında güvenli kişisel dosya senkronizasyonu (Public/Private alanları kullanarak)
- Proje klasörlerini kendi sunucunda tuttuğun bir depoya "tek komutla deploy etme" altyapısı
- Ham HTTP endpointlerini doğrudan açığa çıkarmadan klasörleri taşımak için hafif bir alternatif iş akışı
- Gelecekte geliştirilebilecek GUI tabanlı bir senkronizasyon istemcisi için altyapı

6.2. Kullanım Notları

- Kod içinde gömülü değilse hmac.key ve private_key.pem dosyalarının mevcut olduğundan emin olun.
- Yerel klasörü sisteme bağlamak için önce connect komutunu çalıştırın.
- Kararsız ağ bağlantılarında çok büyük parça boyutları kullanmaktan kaçının (daha küçük parçalar yeniden transfer maliyetini azaltabilir).
- ZIP işlemleri için ortamda PowerShell'in bulunması gerekir.

6.3. Geliştirici YorumuCreator Comments

Projeye başlarken yalnızca basit bir dosya transfer sistemi geliştireceğimi düşünüyordum. Ancak RSA ve AES şifreleme mekanizmalarını uygulamak projeyi beklediğimden çok daha ilginç hale getirdi.

Bu proje üzerinde çalışmak benim için oldukça keyifliydi ve bu tür bir sistemi tek başıma tasarlayıp hayata geçirmek gerçekten tatmin edici bir deneyim oldu.

-Hooah, Famura

7. Credits

- Sadece Famura Projesi

8. Kaynaklar

- NIST. (2001). *Advanced Encryption Standard (AES)*. National Institute of Standards and Technology. https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf
- NIST. (2015). *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM)*. National Institute of Standards and Technology. https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38d.pdf
- Rivest, R., Shamir, A., & Adleman, L. (1978). *A method for obtaining digital signatures and public-key cryptosystems*. Communications of the ACM, 21(2), 120–126. https://doi.org/10.1145/359340.359342
- Rescorla, E. (2000). *HTTP Over TLS*. Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc2818
- Microsoft. (2024). *WinHTTP API documentation*. Microsoft Learn. https://learn.microsoft.com/en-us/windows/win32/winhttp/about-winhttp
- Microsoft. (2024). *Cryptography API: Next Generation (CNG)*. Microsoft Learn. https://learn.microsoft.com/en-us/windows/win32/seccng/cng-portal
- Microsoft. (2024). *Crypt32 API documentation*. Microsoft Learn. https://learn.microsoft.com/en-us/windows/win32/seccrypto/cryptography-functions
- OpenSSL Project. (2024). *OpenSSL Cryptography and SSL/TLS Toolkit*. https://www.openssl.org/docs/
- Microsoft. (2024). *PowerShell Compress-Archive*. Microsoft Learn. https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.archive/compress-archive
- Microsoft. (2024). *PowerShell Expand-Archive*. Microsoft Learn. https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.archive/expand-archive

9. Disk İçeriği

- Projenin kaynak kodları
- Bu raporun .pdf formatında ve İngilizce hali
- Bu raporun .docx formatında ve İngilizce hali
- Bu projenin çalışır hali
- (I Just) Died In Your Arms by Cutting Crew